PROJECT REPORT

AWS VPC PEERING & NETWORK ISOLATION

Private communication between two Amazon Virtual Private Clouds using VPC Peering, route tables, security groups and a bastion host.

Student Name	SHARADA
Batch	Batch 1
Project Title	AWS VPC Peering Project
GitHub Repository	https://github.com/sharadapai08/AWS-VPC-Peering-Project
Course	**Amazon Web Services (AWS) Cloud Computing Internship**
AWS Region	ap-southeast-2 (Sydney) AWS account.
Completion Date	25/10/2026

Abstract

This project demonstrates private communication between two Amazon Web Services (AWS) Virtual Private Clouds (VPCs) using a VPC peering connection. VPC A contains a public bastion host and a private application instance (App-A). VPC B contains a private application instance (App-B). The VPCs use separate, non-overlapping IPv4 ranges. Route tables direct traffic through the peering connection, while security groups restrict inbound SSH and ICMP traffic to specific hosts.

The repository includes terminal evidence for forward and reverse ping tests, SSH access to App-B through a bastion host, an ICMP isolation test, and recovery after the ICMP rule is restored. The evidence shows successful two-way ping tests with 0% packet loss, successful private SSH access, a controlled ping failure with 100% packet loss, and successful recovery with 0% packet loss.

1. Introduction

Amazon VPC allows AWS resources to be launched in logically isolated virtual networks. VPC peering provides a private network route between two VPCs, enabling resources to communicate using private IPv4 addresses when the peering connection is active and the necessary routes and security rules are configured. A peering connection alone does not guarantee connectivity; forward routes, return routes and destination security rules are also required.

2. Project Objectives

• Create two VPCs with non-overlapping CIDR ranges.

• Place the bastion host and App-A in VPC A, and App-B in VPC B.

• Establish a VPC peering connection between VPC A and VPC B.

• Configure subnet associations and route tables for traffic in both directions.

• Restrict inbound SSH and ICMP traffic using security groups.

• Verify private connectivity with ping and SSH tests.

• Demonstrate security-group isolation by removing and restoring the ICMP rule.

3. Architecture and IP Address Plan

The architecture has one public entry point (the bastion host), two private application workloads, and one peering connection. Administrative access follows laptop → bastion → private application host. Application connectivity follows App-A ↔ VPC peering ↔ App-B using private IP addresses.

ewb-vpc-a	VPC A	10.10.0.0/16	First VPC
ewb-a-public	VPC A public subnet	10.10.1.0/24	Bastion subnet
ewb-a-private	VPC A private subnet	10.10.2.0/24	App-A subnet
ewb-vpc-b	VPC B	10.20.0.0/16	Second VPC
ewb-b-private	VPC B private subnet	10.20.1.0/24	App-B subnet
ewb-bastion	ewb-a-public	10.10.1.10 + public IPv4	Administrative jump host
ewb-app-a	ewb-a-private	10.10.2.10	Private application host
ewb-app-b	ewb-b-private	10.20.1.10	Private application host

4. Tools and AWS Resources

• Amazon VPC — virtual network isolation and subnet configuration.
• VPC peering connection — private connectivity between VPC A and VPC B.
• EC2 — bastion host and two application instances.
• Internet Gateway — internet route for the public subnet in VPC A.
• Route tables — direct traffic to local networks, the internet gateway or the peering connection.
• Security groups — control inbound SSH and ICMP traffic.
• SSH ProxyJump — access private instances through the bastion host.
• Ping (ICMP Echo Request) — verify IP connectivity.

5. Implementation Procedure

5.1 Create VPC A and its subnets. Create ewb-vpc-a with CIDR 10.10.0.0/16. Add ewb-a-public (10.10.1.0/24) and ewb-a-private (10.10.2.0/24). Attach ewb-igw-a to VPC A. Associate the public subnet with ewb-rt-a-public and add 0.0.0.0/0 targeting the internet gateway. Associate the private subnet with ewb-rt-a-private and keep only its local route before peering.

5.2 Create VPC B. Create ewb-vpc-b with CIDR 10.20.0.0/16 and the ewb-b-private subnet (10.20.1.0/24). Associate the subnet with ewb-rt-b-private. VPC B does not need an internet gateway for this lab.

5.3 Launch EC2 instances. Launch ewb-bastion in the public subnet with a public IPv4 address, ewb-app-a in VPC A's private subnet, and ewb-app-b in VPC B's private subnet. Use the intended private addresses where available and keep public IPv4 disabled for App-A and App-B.

5.4 Configure security groups. Allow SSH (TCP 22) to the bastion only from the administrator's current public IP /32. Allow SSH to both private hosts from the bastion's private IP (10.10.1.10/32). Allow ICMP Echo Request to App-A from App-B (10.20.1.10/32) and to App-B from App-A (10.10.2.10/32). Do not expose SSH or ICMP to 0.0.0.0/0.

6. VPC Peering and Route Tables

Create a peering connection named ewb-a-b-peer with VPC A as requester and VPC B as accepter. Accept the request and wait until the status becomes Active. Record the actual peering connection ID from the AWS console.

Route table	Destination	Target	Purpose
ewb-rt-a-private	10.10.0.0/16	local	Traffic inside VPC A
ewb-rt-a-private	10.20.0.0/16	Peering ID (pcx-…)	App-A to VPC B
ewb-rt-b-private	10.20.0.0/16	local	Traffic inside VPC B
ewb-rt-b-private	10.10.0.0/16	Peering ID (pcx-…)	Return path to VPC A
ewb-rt-a-public	10.10.0.0/16	local	Bastion to App-A
ewb-rt-a-public	0.0.0.0/0	Internet Gateway	Bastion internet access
ewb-rt-a-public	10.20.0.0/16	Peering ID (pcx-…)	Bastion to App-B

Both VPCs require routes because network communication is bidirectional: the destination must be able to send replies back to the source. The public route table in VPC A also needs a route to VPC B so that ProxyJump can reach App-B from the bastion.

7. Security Group Rules

Security group	Inbound type	Source
ewb-sg-bastion	SSH / TCP 22	Administrator's public IPv4 /32
ewb-sg-app-a	SSH / TCP 22	10.10.1.10/32 (bastion)
ewb-sg-app-a	ICMP Echo Request	10.20.1.10/32 (App-B)
ewb-sg-app-b	SSH / TCP 22	10.10.1.10/32 (bastion)
ewb-sg-app-b	ICMP Echo Request	10.10.2.10/32 (App-A)

Security groups are stateful. For this project, default outbound allow rules and default network ACLs are assumed, as described in the EWB workbook. If actual configurations differ, update this report to match the AWS console.

8. Testing and Verification

The uploaded repository contains screenshots for forward and reverse ping, SSH ProxyJump, ICMP failure and recovery. The results below are transcribed from the visible terminal evidence.

Test	Command / Action	Observed result in screenshot	Status
Forward path	App-A: ping -c 4 10.20.1.10	4 packets transmitted, 4 received, 0% packet loss	Pass
Reverse path	App-B: ping -c 4 10.10.2.10	4 packets transmitted, 4 received, 0% packet loss	Pass
Private SSH	Laptop: ssh ewb-app-b	Amazon Linux 2023 shell opened on 10.20.1.10 thr	ough ProxyJumpPass
Isolation	Remove App-B ICMP inbound rule, pi	g from App-A4 packets transmitted, 0 received, 100% packet loss	Pass (expected blo
Recovery	Restore App-B ICMP inbound rule, re	eat ping4 packets transmitted, 4 received, 0% packet oss	Pass
Pre-peering baseline	App-A: ping -c 4 10.20.1.10 before rou	tes/peeringA distinct baseline screenshot was not clearly inclu	ed in the IPEvidence missing

8.1 Active peering evidence

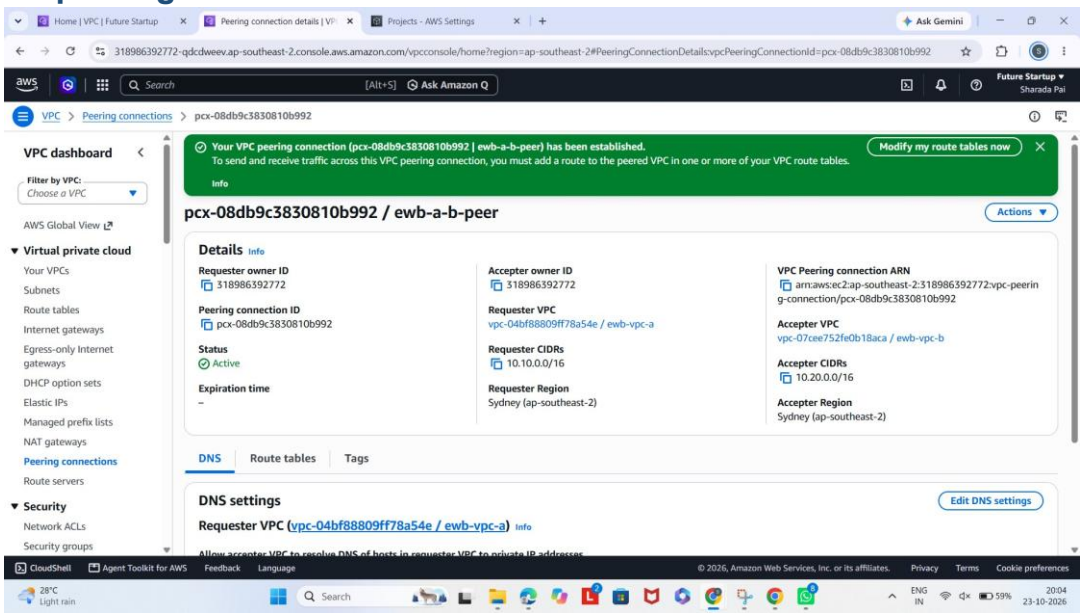


Figure 1. Screenshot file is named as a pre-peering baseline, but the image appears to show the VPC peering console with Active status. It is therefore treated as peering-status evidence, not baseline-failure evidence.

8.2 Forward path: App-A to App-B

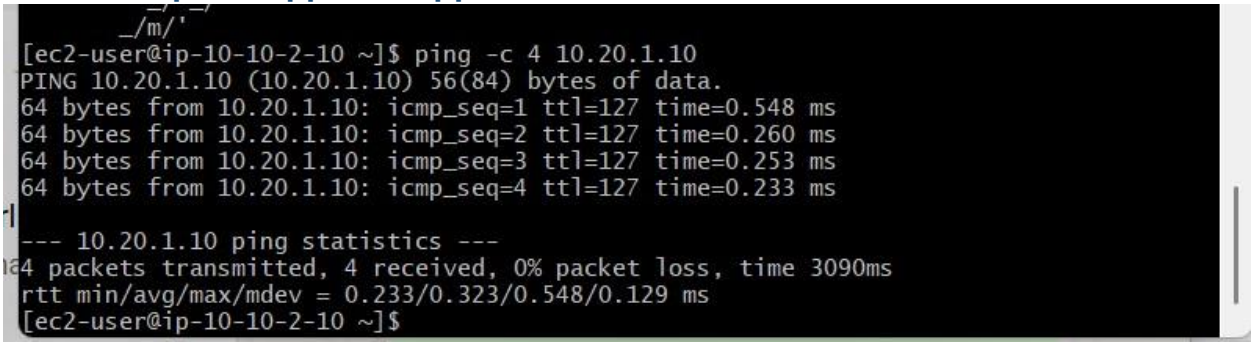


Figure 2. App-A pinging 10.20.1.10: four replies and 0% packet loss.

8.3 Reverse path: App-B to App-A

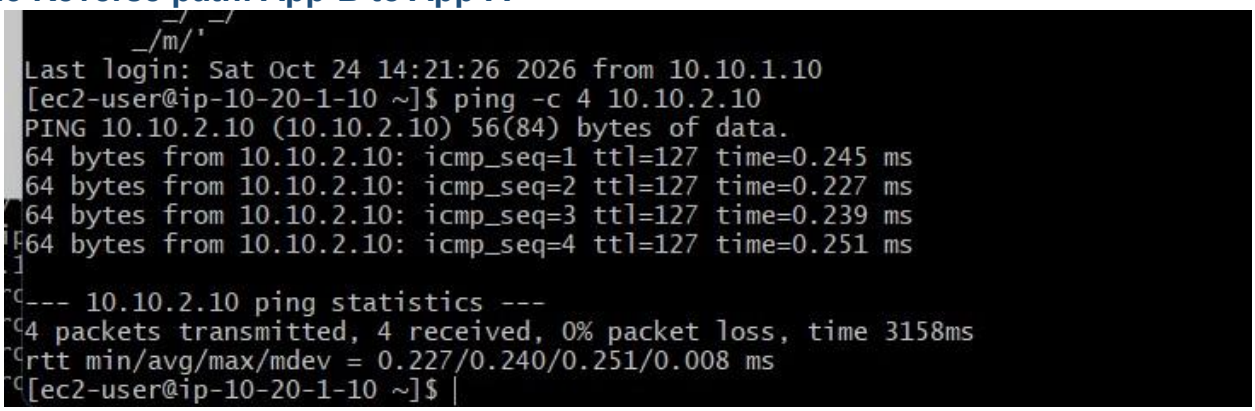


Figure 3. App-B pinging 10.10.2.10: four replies and 0% packet loss.

8.4 Private SSH through the bastion

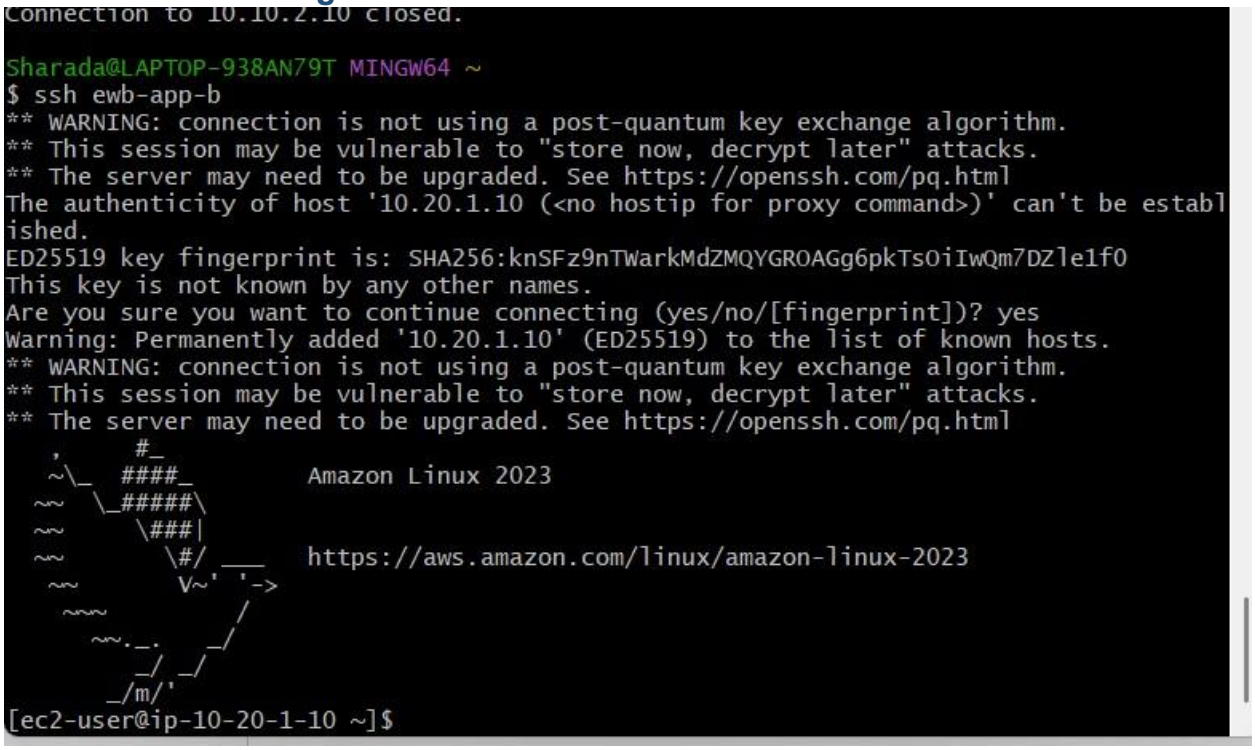


Figure 4. SSH ProxyJump reaches the Amazon Linux 2023 shell on App-B at 10.20.1.10.

8.5 Controlled security-group failure

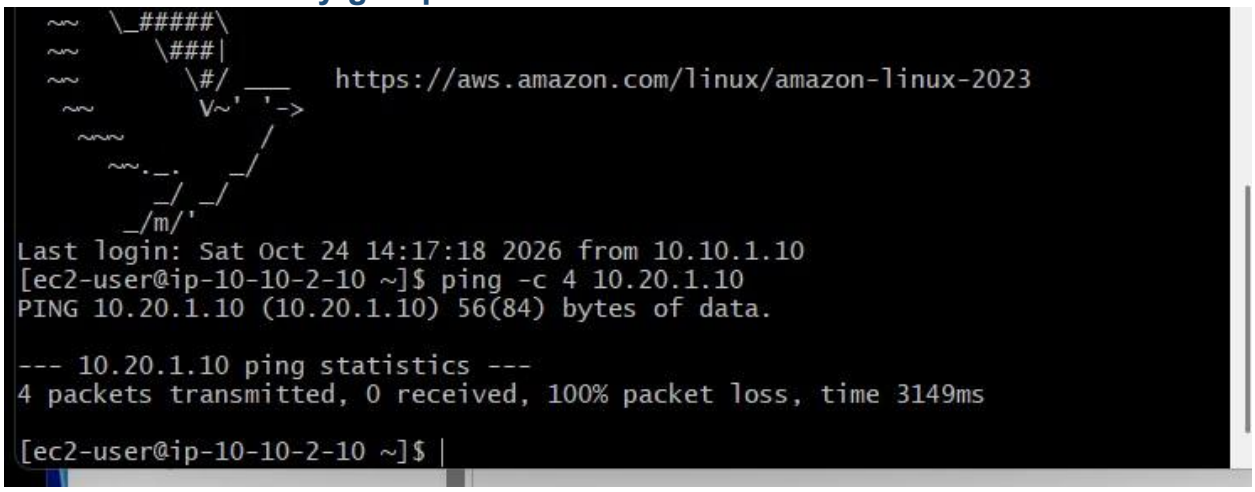


Figure 5. After the ICMP Echo Request rule is removed from App-B's security group, the ping test shows 100% packet loss.

8.6 Recovery after restoring the rule

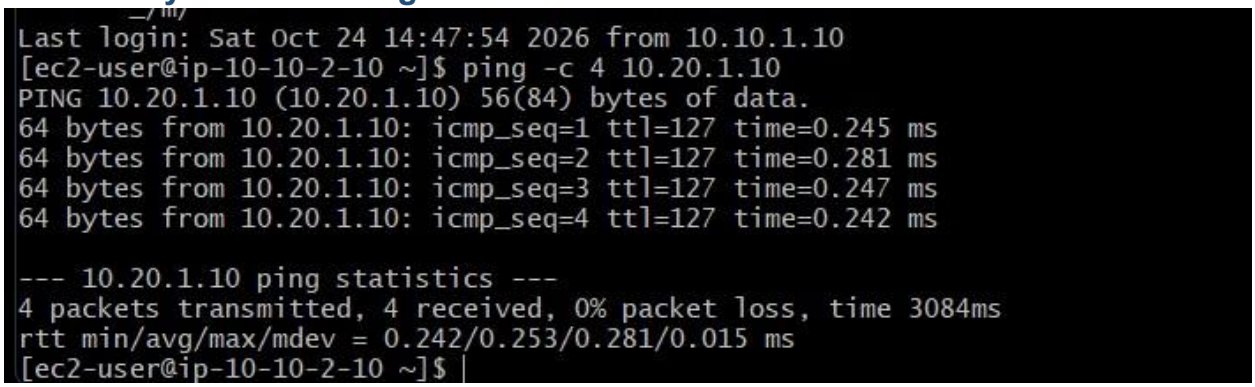


Figure 6. After the ICMP rule is restored, the ping test succeeds with 0% packet loss.

9. Results and Discussion

The submitted terminal screenshots support the conclusion that the two private EC2 instances can exchange ICMP traffic through the VPC peering connection after routes and security group rules are configured. The forward and reverse tests each show four packets received and 0% packet loss. The SSH screenshot shows access to App-B through the bastion host. The controlled failure test shows 100% packet loss when the ICMP rule is removed, and the recovery screenshot shows 0% packet loss after the rule is restored.

The evidence demonstrates that a working peering connection needs more than an Active status: route tables must direct traffic to the peering connection, return routes must exist, and security groups must permit the intended traffic.

10. Conclusion

This project demonstrates how AWS VPC Peering enables private communication between workloads in separate VPCs. The implementation combines non-overlapping CIDR blocks, public and private subnets, a bastion host, EC2 instances, VPC peering, route tables and security groups. The successful ping and SSH screenshots, together with the controlled failure and recovery evidence, illustrate both connectivity and access control. Before final submission, the missing pre-peering baseline screenshot should be added if available, and actual resource IDs and final cleanup evidence should be included.

11. Cleanup Checklist

- Save all screenshots and record resource IDs before deleting resources.
- Terminate the three lab EC2 instances and verify lab EBS volumes/snapshots.
- Release any Elastic IP created for the lab.
- Delete the peering connection and its peer routes.
- Delete lab security groups, subnets, custom route tables and the internet gateway after dependencies areremoved.
- Delete both lab VPCs and verify that no unwanted chargeable resources remain.
- Add a screenshot or note showing cleanup completion.

12. References

1. Student GitHub repository: https://github.com/sharadapai08/AWS-VPC-Peering-Project
2. EWB Cloud Labs, AWS VPC Peering Student Project Workbook, October 2026.
3. AWS VPC Peering documentation: https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html
4. AWS route table documentation: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html